

Analysis of PennSound speech-to-text

includes additional analysis

```
1 using DataFrames, CSV, Statistics, Dates, Plots, Distributions, RollingFunctions, GLM, StatsBase, StatsPlots
```

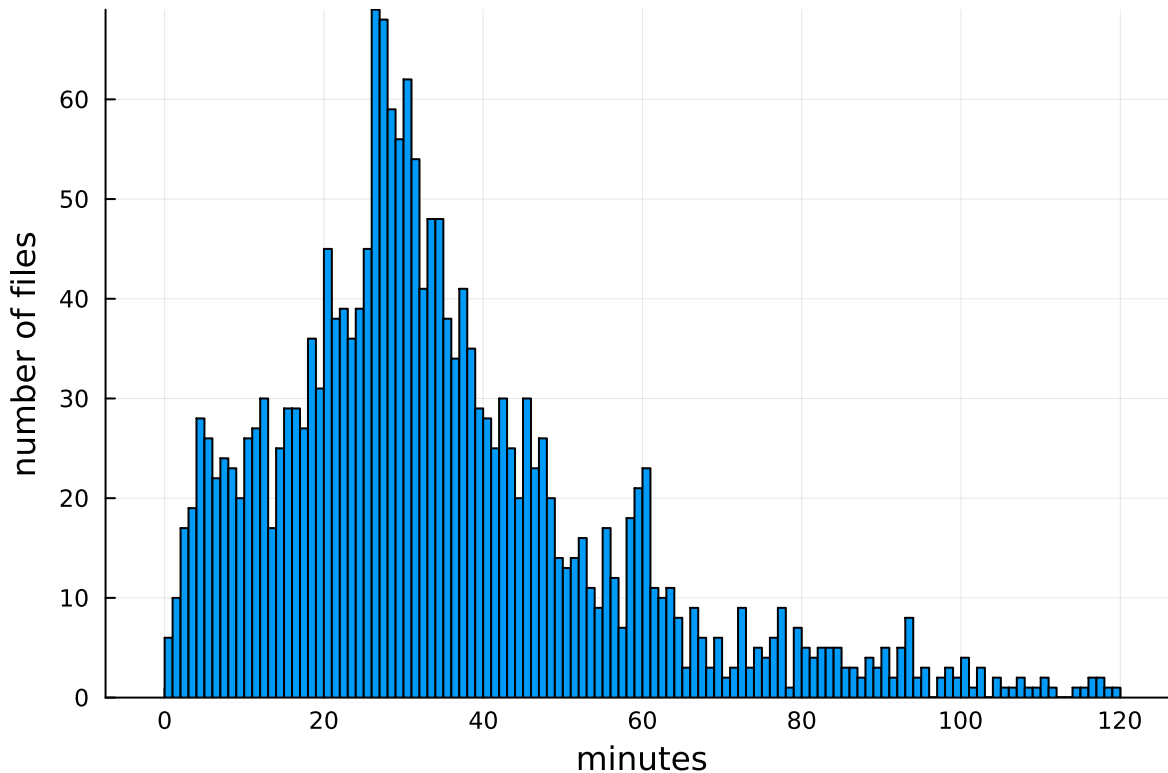
Full Corpus Estimate

This set of files, downloaded from the website, is a majority of the complete recordings, representing about 1300 hours. The full set of audio files on the website is actually several thousand hours more, but there are a large number of files that are excerpts from the complete recordings (e.g. individual poems).

pennsound_durations =

duration	
1	163.735
2	1643.23
3	274.102
4	2835.54
5	1184.65
6	1327.38
7	3643.33
8	1888.83
9	321.357
10	746.587
more	
2049	1566.77

```
1 pennsound_durations = CSV.read("pennsound_durations.tsv", DataFrame, delim="\t")
2
```



```
1 histogram(pennsound_durations.duration ./ 60, bins=0:120, label=:none,
  xlabel="minutes", ylabel="number of files")
```

(37, 1)

```
1 size(pennsound_durations[pennsound_durations.duration .> 7200,:])
```

Sample Construction

A random list of 100 files with the word *Complete* were selected, and the two following files were added from outside the Authors' page:

- Clay-Steve_Close-Listening_5-17-21
- PoemTalk-198_On-three-Larry-Price-poems

Some files were very short or had very little speech, so a second random list of 100 files was created. From this list, these 12 files were drawn alphabetically to return the list to length 100:

- Ashbery-John_Complete-Reading_Contemporary-American-Poetry...
- Ashbery-John_Complete-Recording_St-Marks_NY
- Auster-Paul_Complete-Reading_UB_4-26-95
- Bellamy-Dodie_Complete-Reading_A-Voice-Box_Canessa-Park_1-17-09
- Benson-Steve_Complete-Recording_Views-of-Communist-China_1220...
- Berkson-Bill_Complete-Reading_Poetry-Project_St-Marks-NY_5-17-78
- Berssenbrugge-Mei-Mei_Complete-Reading_UB_9-20-00

- Berssenbrugge-Mei-Mei_Complete-Recording_Segue-DH_NYC_2-21-98
- Bromige-David_Complete-Reading_Albany-CA_Fall-1968
- Bromige-David_Complete-Recording_New-Poems_Unknown-Reel
- Browne-Laynie_Complete-Reading_Contemporary-Writers-Series...
- Brown-Lee-Ann_Complete-Reading_Tender-Buttons-Press_UPenn_10-21-03

Therefore, the sample comes from a near-random selection of 100 complete recordings available from the website. For each file, a small excerpt was taken for analysis, based on speech activity detection (SAD) labels. Starting from the midpoint of each file, SAD labels were read until 5 minutes of speech had accumulated. These labels then determined the bounds of the excerpts, after padding with silence. The following table describes the excerpts in terms of offset into the original file and duration of the excerpt. The table also gives the short name used throughout for the excerpt.

excerpts =

	file	excerpt	offset	duration
1	"Andrews-Bruce-and-Charles-North_Compl	"AndrewsEx1"	2412.57	585.15
2	"Antin-David_Complete_Seminar_Universi	"AntinEx1"	4514.15	462.47
3	"Ashbery-John_01_Complete-Reading_WBAI	"Ashbery1Ex1"	981.23	432.82
4	"Ashbery-John_Complete-Reading_Contemp	"Ashbery2Ex1"	595.46	446.79
5	"Ashbery-John_Complete-Recording_Attit	"Ashbery3Ex1"	860.9	390.93
6	"Ashbery-John_Complete-Recording_Honor	"Ashbery4Ex1"	1647.89	458.35
7	"Ashbery-John_Complete-Recording_Pione	"Ashbery5Ex1"	2320.08	393.12
8	"Ashbery-John_Complete-Recording_St-Ma	"Ashbery6Ex1"	1312.54	466.21
9	"Ashbery-John_Complete-Recording_The-S	"Ashbery7Ex1"	1850.73	440.09
10	"Ashbery-John_Complete-Recording_WBAI-	"Ashbery8Ex1"	919.02	458.39
more				
100	"Yau-John_02_Complete-Reading_SUNY-Buf	"YauEx1"	1444.67	379.11

```
1 excerpts = CSV.read("excerpts.tsv", DataFrame, delim="\t")
2
437.486500000000004
1 mean(excerpts.duration)
81.91142522612923
1 std(excerpts.duration)

total hours of audio
```

12.152402777777779

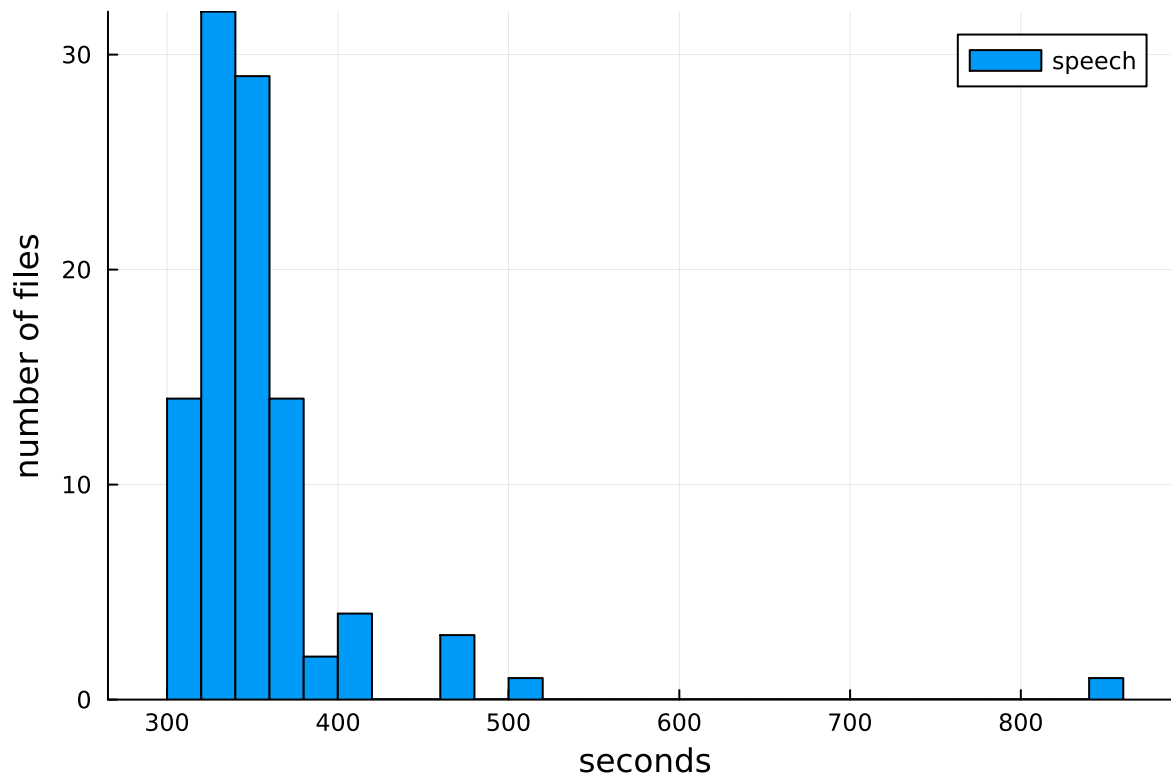
```
1 sum(excerpts.duration) / 3600
```

The original goal was a random sample of 100 clips, each 5 minutes in duration, which would be 8.3 hours of audio if done exactly that way. As described above, SAD labels were used to locate 5 minutes of speech, meaning the excerpts are longer than 5 minutes. The total audio duration is 12.15 hours, which would suggest about one third of the audio is non-speech. However, the reference transcripts tend to indicate more than 5 minutes of speech, due to speech undetected by SAD, overlapping speech (represented by transcribers but not by SAD), or possibly human error (segments padded with silence). Summing the segment lengths from the human transcripts gives a total of 9.84 hours of speech, so the amount of non-speech is closer to one fifth of the audio. The table and histogram below illustrate the amount of speech per file in seconds.

speech =

	file	speech
1	"AndrewsEx1"	341.623
2	"AntinEx1"	372.994
3	"Ashbery1Ex1"	343.004
4	"Ashbery2Ex1"	348.673
5	"Ashbery3Ex1"	300.015
6	"Ashbery4Ex1"	361.438
7	"Ashbery5Ex1"	315.977
8	"Ashbery6Ex1"	354.344
9	"Ashbery7Ex1"	336.321
10	"Ashbery8Ex1"	363.231
more		
100	"YauEx1"	337.962

```
1 speech = CSV.read("speech.tsv", DataFrame, delim="\t")
2
```



```
1 histogram(speech[:, :speech], label="speech", xlabel="seconds", ylabel="number of files")
```

```
354.16752999999994
```

```
1 mean(speech.speech)
```

```
61.48031766484953
```

```
1 std(speech.speech)
```

file	speech
1 "GinsbergEx1"	858.552

```
1 speech[speech.speech > 800, :]
```

```
9.837986944444443
```

```
1 sum(speech.speech) / 3600
```

Word Error Rates

Here we load in raw WERs. We have separate tables for the WER as well as the substitution, deletion, and insertion components.

wers =

	file	aws	azure	google	ibm	nemo	rev	whisper	whisper
1	"AndrewsEx1"	29.0	23.1	24.7	33.4	18.8	19.0	19.1	21.6
2	"AntinEx1"	15.7	18.1	16.4	21.2	20.3	15.0	19.4	21.2
3	"Ashbery1Ex1"	3.5	4.3	4.2	5.8	3.4	3.7	2.8	3.5
4	"Ashbery2Ex1"	3.6	4.6	4.2	6.6	4.4	3.2	3.8	4.6
5	"Ashbery3Ex1"	3.9	4.8	5.5	6.2	4.9	3.2	4.7	5.3
6	"Ashbery4Ex1"	14.6	14.9	15.6	17.4	18.2	15.6	18.1	17.2
7	"Ashbery5Ex1"	5.4	6.3	4.8	9.1	5.7	3.2	3.9	4.7
8	"Ashbery6Ex1"	2.0	2.4	4.1	3.6	2.8	2.4	2.0	6.4
9	"Ashbery7Ex1"	4.1	4.8	5.2	11.4	3.5	4.2	3.0	3.6
10	"Ashbery8Ex1"	4.5	4.5	4.6	8.6	4.7	4.2	3.5	5.8
more									
100	"YauEx1"	5.5	6.0	7.4	10.7	4.4	4.5	3.6	4.9

```
1 wers = CSV.read("wer.tsv", DataFrame, delim="\t")
2
```

	file	aws	azure	google	ibm	nemo	rev	whisper	whisperc
1	"BonvicinoEx1"	26.6	24.1	27.7	30.4	26.4	21.5	24.0	22.5

```
1 wers[wers.file .== "BonvicinoEx1",:]
```

substitution error rates only

wers_s =

	file	aws	azure	google	ibm	nemo	rev	whisper	whisper
1	"AndrewsEx1"	20.1	16.7	16.1	26.7	14.3	14.0	13.2	14.4
2	"AntinEx1"	7.5	4.6	5.6	9.7	5.4	5.9	4.8	4.0
3	"Ashbery1Ex1"	2.5	2.7	2.1	4.6	2.1	2.5	1.6	1.5
4	"Ashbery2Ex1"	2.1	3.0	2.3	4.9	1.3	1.6	1.3	1.4
5	"Ashbery3Ex1"	1.7	2.0	1.9	3.9	1.4	1.2	0.9	1.6
6	"Ashbery4Ex1"	6.8	5.9	6.2	9.3	4.7	7.2	5.4	5.4
7	"Ashbery5Ex1"	3.9	4.6	3.5	6.7	2.6	2.1	2.2	2.7
8	"Ashbery6Ex1"	1.3	1.5	1.7	3.1	0.8	1.3	0.8	1.3
9	"Ashbery7Ex1"	2.9	3.9	3.2	9.4	2.1	3.1	2.3	2.5
10	"Ashbery8Ex1"	3.4	3.6	2.9	6.9	3.1	3.3	2.8	2.4
more									
100	"YauEx1"	4.3	3.8	5.5	8.5	2.5	3.4	2.5	3.5

```
1 wers_s = CSV.read("wer_s.tsv", DataFrame, delim="\t")
2
```

deletion error rates only

wers_d =

	file	aws	azure	google	ibm	nemo	rev	whisper	whisper
1	"AndrewsEx1"	3.5	1.3	3.9	2.9	1.2	1.9	1.9	2.3
2	"AntinEx1"	6.9	12.8	9.7	10.7	13.1	6.4	10.9	13.7
3	"Ashbery1Ex1"	0.4	1.0	1.5	0.6	0.7	0.8	0.5	1.6
4	"Ashbery2Ex1"	0.8	0.8	1.2	0.9	2.3	0.7	1.7	1.8
5	"Ashbery3Ex1"	1.6	2.6	3.4	2.3	3.0	1.9	3.5	3.6
6	"Ashbery4Ex1"	5.3	7.0	6.4	5.9	12.5	5.7	11.5	10.5
7	"Ashbery5Ex1"	0.3	0.9	0.6	1.7	2.8	0.7	1.2	1.3
8	"Ashbery6Ex1"	0.0	0.4	1.9	0.1	1.8	0.7	0.5	4.8
9	"Ashbery7Ex1"	0.3	0.5	1.2	1.1	0.6	0.6	0.2	0.7
10	"Ashbery8Ex1"	0.4	0.4	1.0	0.9	1.1	0.4	0.3	2.6
more									
100	"YauEx1"	0.5	1.6	0.7	1.6	1.7	0.7	1.1	1.1

```
1 wers_d = CSV.read("wer_d.tsv", DataFrame, delim="\t")
2
```

insertion error rates only

wers_i =

	file	aws	azure	google	ibm	nemo	rev	whisper	whisper
1	"AndrewsEx1"	5.4	5.0	4.8	3.8	3.3	3.0	4.0	4.9
2	"AntinEx1"	1.3	0.6	1.2	0.7	1.8	2.7	3.7	3.5
3	"Ashbery1Ex1"	0.6	0.5	0.6	0.6	0.5	0.5	0.8	0.4
4	"Ashbery2Ex1"	0.7	0.9	0.8	0.8	0.9	0.9	0.8	1.4
5	"Ashbery3Ex1"	0.6	0.2	0.2	0.0	0.5	0.1	0.3	0.1
6	"Ashbery4Ex1"	2.6	2.1	3.0	2.2	0.9	2.7	1.2	1.3
7	"Ashbery5Ex1"	1.1	0.8	0.8	0.7	0.3	0.4	0.4	0.7
8	"Ashbery6Ex1"	0.7	0.5	0.5	0.4	0.2	0.4	0.7	0.3
9	"Ashbery7Ex1"	0.9	0.5	0.8	0.9	0.7	0.5	0.5	0.5
10	"Ashbery8Ex1"	0.7	0.5	0.7	0.8	0.5	0.4	0.4	0.8
more									
100	"YauEx1"	0.7	0.6	1.1	0.6	0.2	0.3	0.0	0.2

```
1 wers_i = CSV.read("wer_i.tsv", DataFrame, delim="\t")
2
```

sort by rev, the best performer

wers_sorted_by_rev =

	file	aws	azure	google	ibm	nemo	rev	whisper	whisp
1	"Bromige2Ex1"	2.4	3.0	2.0	4.2	2.4	1.1	2.0	1.9
2	"DuPlessis1Ex1"	3.0	2.9	2.9	4.4	3.9	1.8	1.9	5.4
3	"DuPlessis2Ex1"	2.8	2.6	3.6	6.7	2.9	2.0	1.8	1.6
4	"GladmanEx1"	2.2	2.4	3.2	4.1	4.2	2.2	1.9	10.9
5	"JarnotEx1"	4.3	4.0	4.5	9.3	5.5	2.3	3.1	4.3
6	"Ashbery6Ex1"	2.0	2.4	4.1	3.6	2.8	2.4	2.0	6.4
7	"Robinson1Ex1"	3.0	2.8	5.0	4.2	4.1	2.5	2.6	3.6
8	"RichardsEx1"	3.9	3.9	2.8	8.2	2.8	2.8	3.4	7.1
9	"Robinson3Ex1"	3.6	3.2	3.9	5.2	4.2	2.9	2.9	5.3
10	"Ashbery2Ex1"	3.6	4.6	4.2	6.6	4.4	3.2	3.8	4.6
more									
100	"GinsbergEx1"	37.2	36.3	36.2	44.0	34.4	33.3	32.4	39.4

```
1 wers_sorted_by_rev = sort(wers, [:rev])
```

add mean WER to table

wers_with_mean =

	file	aws	azure	google	ibm	nemo	rev	whisper	more
1	"AndrewsEx1"	29.0	23.1	24.7	33.4	18.8	19.0	19.1	
2	"AntinEx1"	15.7	18.1	16.4	21.2	20.3	15.0	19.4	
3	"Ashbery1Ex1"	3.5	4.3	4.2	5.8	3.4	3.7	2.8	
4	"Ashbery2Ex1"	3.6	4.6	4.2	6.6	4.4	3.2	3.8	
5	"Ashbery3Ex1"	3.9	4.8	5.5	6.2	4.9	3.2	4.7	
6	"Ashbery4Ex1"	14.6	14.9	15.6	17.4	18.2	15.6	18.1	
7	"Ashbery5Ex1"	5.4	6.3	4.8	9.1	5.7	3.2	3.9	
8	"Ashbery6Ex1"	2.0	2.4	4.1	3.6	2.8	2.4	2.0	
9	"Ashbery7Ex1"	4.1	4.8	5.2	11.4	3.5	4.2	3.0	
10	"Ashbery8Ex1"	4.5	4.5	4.6	8.6	4.7	4.2	3.5	
more									
100	"YauEx1"	5.5	6.0	7.4	10.7	4.4	4.5	3.6	

```
1 wers_with_mean = transform(wers, AsTable([:aws, :azure, :google, :ibm, :nemo, :rev, :whisper, :whispercpp]) => ByRow(mean) => :mean)
```

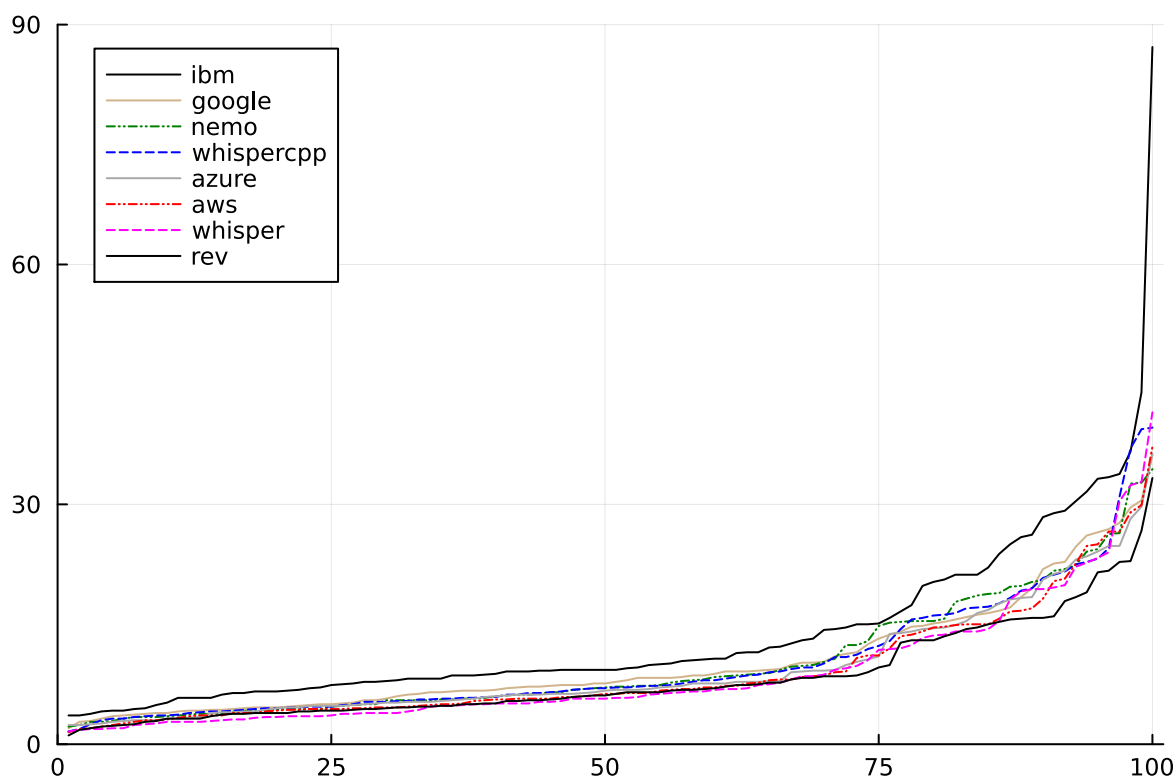
sort by mean WER

wers_sorted_by_mean =

	file	aws	azure	google	ibm	nemo	rev	whisper
1	"Bromige2Ex1"	2.4	3.0	2.0	4.2	2.4	1.1	2.0
2	"DuPlessis2Ex1"	2.8	2.6	3.6	6.7	2.9	2.0	1.8
3	"Ashbery6Ex1"	2.0	2.4	4.1	3.6	2.8	2.4	2.0
4	"DuPlessis1Ex1"	3.0	2.9	2.9	4.4	3.9	1.8	1.9
5	"Robinson1Ex1"	3.0	2.8	5.0	4.2	4.1	2.5	2.6
6	"Berssenbrugge1Ex1"	1.9	5.5	3.7	6.4	3.4	3.9	2.4
7	"GladmanEx1"	2.2	2.4	3.2	4.1	4.2	2.2	1.9
8	"Ashbery1Ex1"	3.5	4.3	4.2	5.8	3.4	3.7	2.8
9	"Robinson3Ex1"	3.6	3.2	3.9	5.2	4.2	2.9	2.9
10	"AusterEx1"	2.6	2.5	8.3	3.8	2.2	4.8	3.3
more								
100	"GinsbergEx1"	37.2	36.3	36.2	44.0	34.4	33.3	32.4

```
1 wers_sorted_by_mean = sort(wers_with_mean, :mean)
```

plot WERs, each system sorted separately



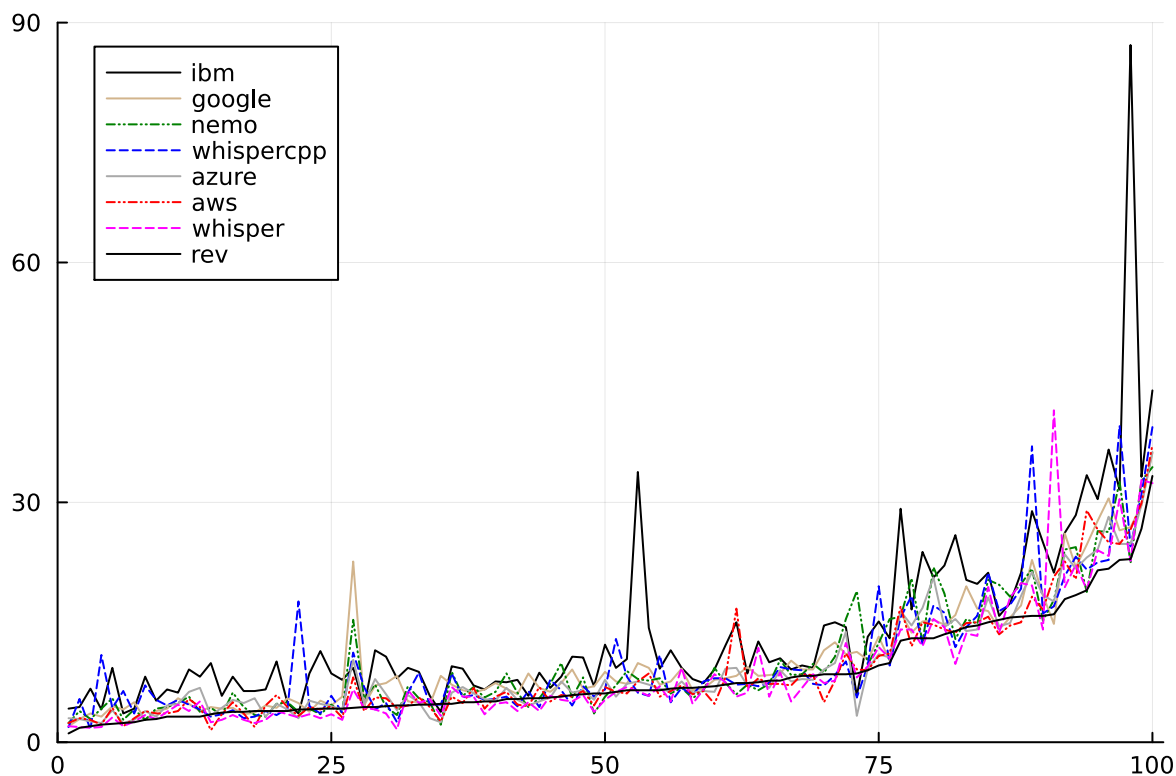
```

1 begin
2   plot(sort(wers.ibm),label="ibm", linestyle=:solid, color=:black)
3   plot!(sort(wers.google),label="google", linestyle=:solid, color=:tan)
4   plot!(sort(wers.nemo),label="nemo", linestyle=:dashdotdot, color=:green)
5   plot!(sort(wers.whispercpp),label="whispercpp", linestyle=:dash, color=:blue)
6   plot!(sort(wers.azure),label="azure", linestyle=:solid, color=:darkgray)
7   plot!(sort(wers.aws),label="aws",xlim=(0,101),ylim=(0,90),
8         linestyle=:dashdotdot,color=:red)
9   plot!(sort(wers.whisper),label="whisper", linestyle=:dash, color=:magenta)
10  plot!(sort(wers.rev),label="rev", linestyle=:solid,color=:black)
11 end

```

plot WERs, sorted by rev (best system)

note rev is the bottom black line, smoothly ascending

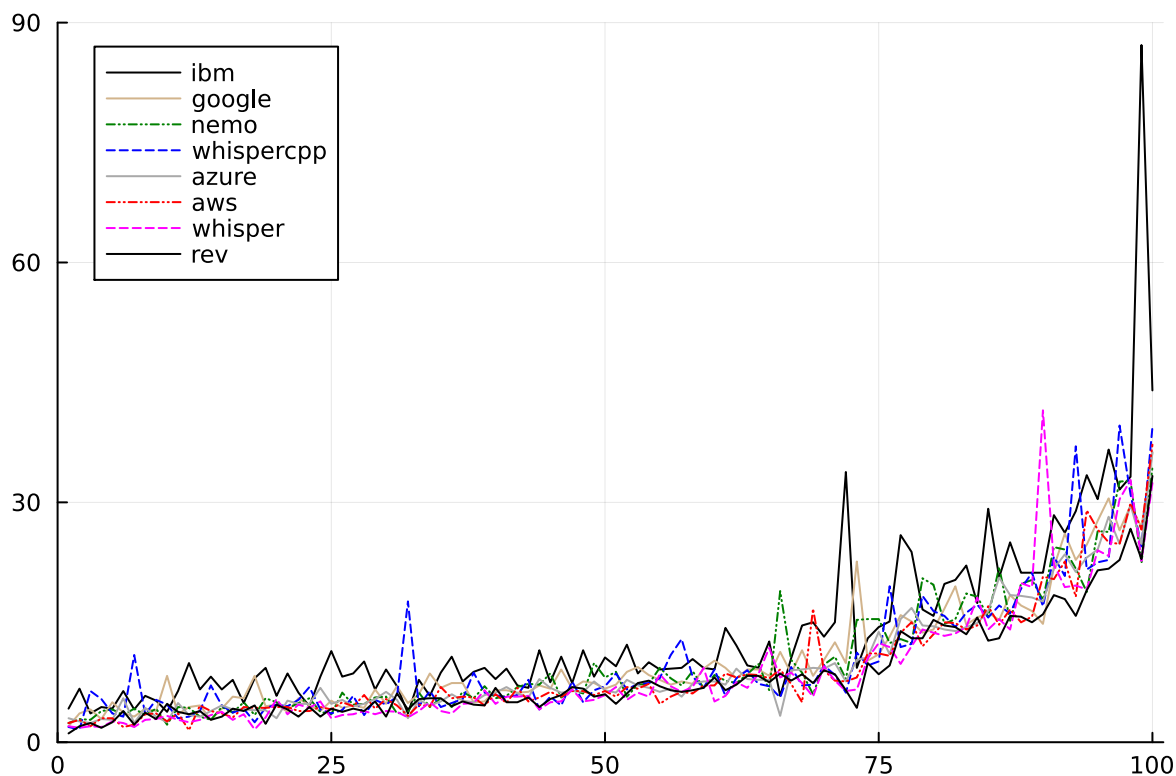


```

1 begin
2   w1 = wers_sorted_by_rev
3   plot(w1.ibm,label="ibm", linestyle=:solid, color=:black)
4   plot!(w1.google,label="google", linestyle=:solid, color=:tan)
5   plot!(w1.nemo,label="nemo", linestyle=:dashdotdot, color=:green)
6   plot!(w1.whispercpp,label="whispercpp", linestyle=:dash, color=:blue)
7   plot!(w1.azure,label="azure", linestyle=:solid, color=:darkgray)
8   plot!(w1.aws,label="aws",xlim=(0,101),ylim=(0,90),
9         linestyle=:dashdotdot,color=:red)
10  plot!(w1.whisper,label="whisper", linestyle=:dash, color=:magenta)
11  plot!(w1.rev,label="rev", linestyle=:solid,color=:black)
12 end

```

WERs sorted by mean WER



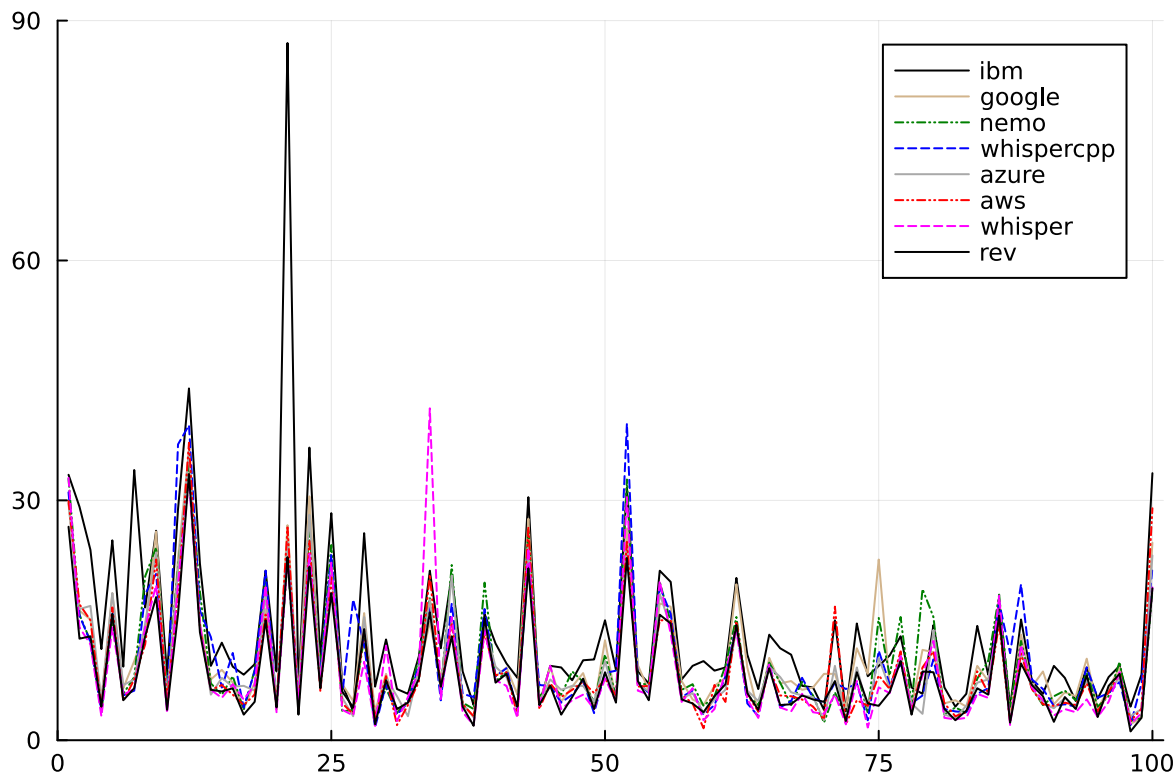
```

1 begin
2   w2 = wers_sorted_by_mean
3   plot(w2.ibm,label="ibm", linestyle=:solid, color=:black)
4   plot!(w2.google,label="google", linestyle=:solid, color=:tan)
5   plot!(w2.nemo,label="nemo", linestyle=:dashdotdot, color=:green)
6   plot!(w2.whispercpp,label="whispercpp", linestyle=:dash, color=:blue)
7   plot!(w2.azure,label="azure", linestyle=:solid, color=:darkgray)
8   plot!(w2.aws,label="aws",xlim=(0,101),ylim=(0,90),
9         linestyle=:dashdotdot,color=:red)
10  plot!(w2.whisper,label="whisper", linestyle=:dash, color=:magenta)
11  plot!(w2.rev,label="rev", linestyle=:solid,color=:black)
12 end

```

plot WERs sorted by IBM provided snr

it's perhaps slightly true that higher SNR yields lower WER

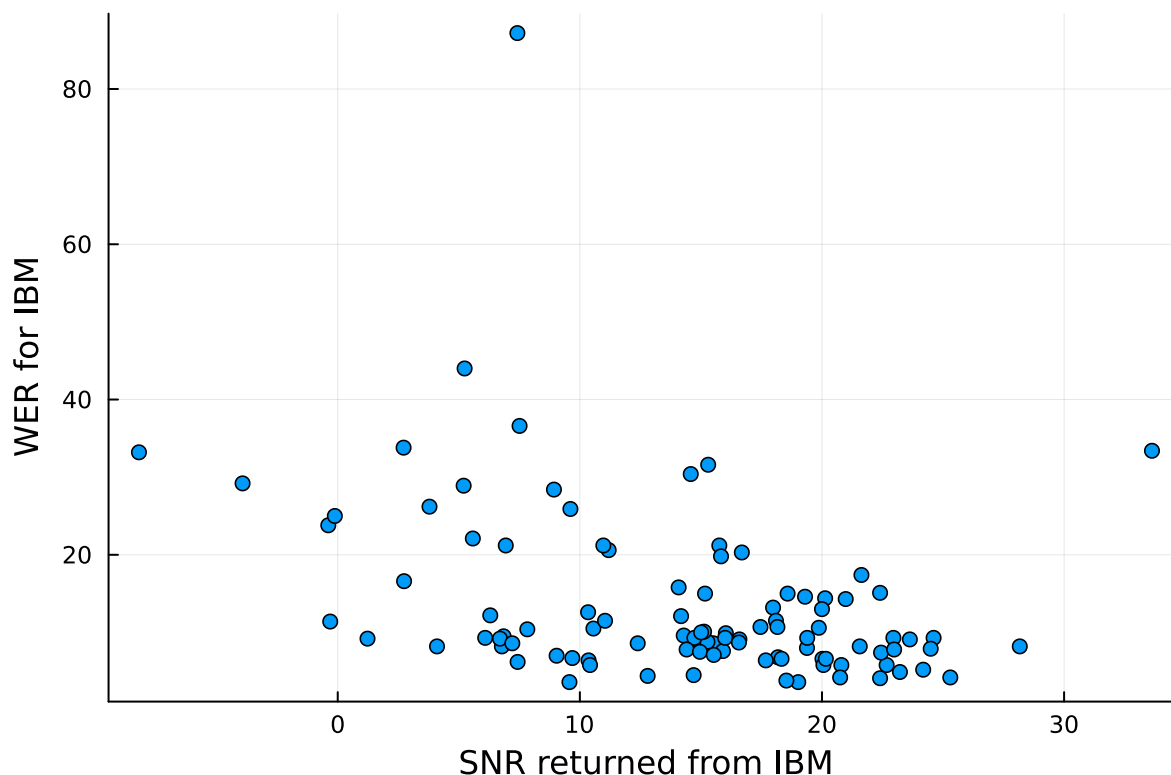


```

1 begin
2   wers_sorted_by_snr = sort(wers, [:snr])
3   plot(wers_sorted_by_snr.ibm,label="ibm", linestyle=:solid, color=:black)
4   plot!(wers_sorted_by_snr.google,label="google", linestyle=:solid, color=:tan)
5   plot!(wers_sorted_by_snr.nemo,label="nemo", linestyle=:dashdotdot, color=:green)
6   plot!(wers_sorted_by_snr.whispercpp,label="whispercpp", linestyle=:dash,
7     color=:blue)
8   plot!(wers_sorted_by_snr.azure,label="azure", linestyle=:solid, color=:darkgray)
9   plot!(wers_sorted_by_snr.aws,label="aws",xlim=(0,101),ylim=(0,90),
10    linestyle=:dashdotdot,color=:red)
11  plot!(wers_sorted_by_snr.whisper,label="whisper", linestyle=:dash,
12    color=:magenta)
13  plot!(wers_sorted_by_snr.rev,label="rev", linestyle=:solid,color=:black)
14 end

```

The SNR values are produced by IBM, so let's look at just that system



```
1 begin
2   scatter(wers.snr, wers.ibm, label=:none, xlabel="SNR returned from
   IBM",ylabel="WER for IBM")
3 end
```

```
snr_model =
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}}, GLM.DensePredCho
```

```
mean ~ 1 + snr
```

Coefficients:

	Coef.	Std. Error	t	Pr(> t)	Lower 95%	Upper 95%
(Intercept)	14.8973	1.47639	10.09	<1e-16	11.9674	17.8271
snr	-0.350733	0.0923513	-3.80	0.0003	-0.534001	-0.167464

```
1 snr_model = lm(@formula(mean ~ snr), wers_with_mean)
```

```
[14.8973, -0.350733]
```

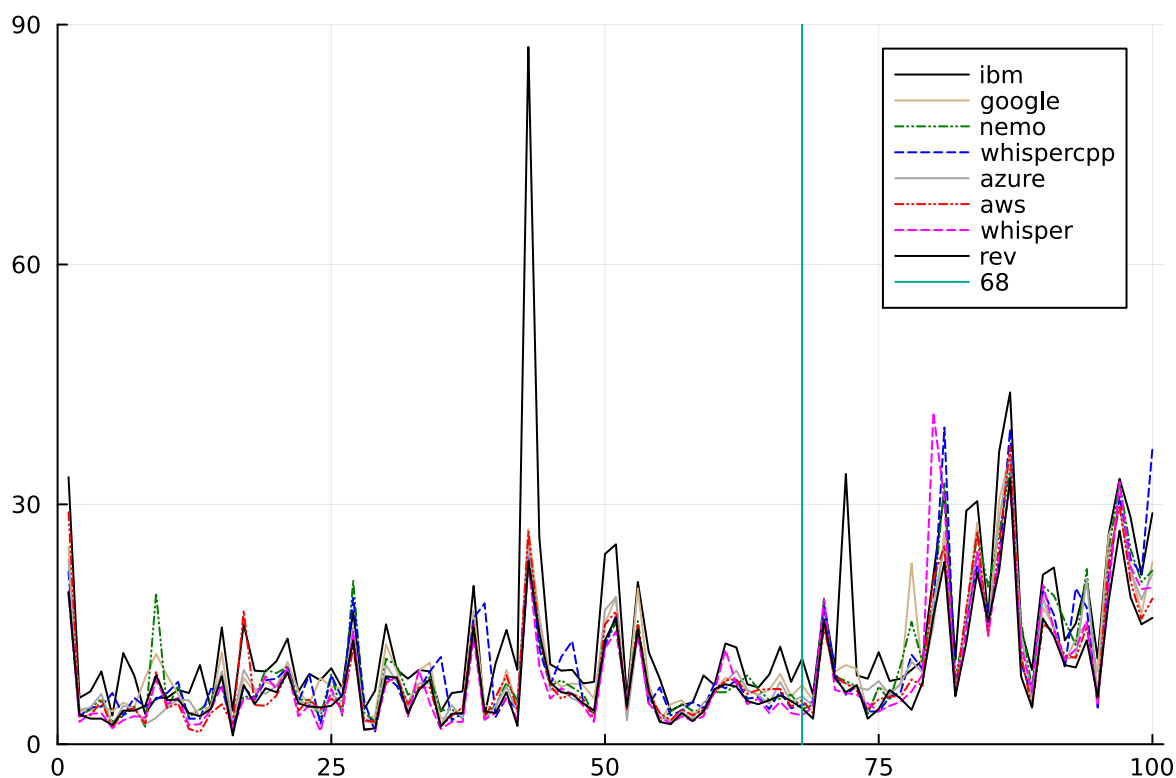
```
1 xx, yy = snr_model.model.pp.beta0
```

68 recordings have one speaker, 32 recordings have multiple speakers

68

```
1 count(wers.nsp .== 1)
```

plot WERs sorted by number of speakers with line at 68: left hand side has one speaker, right hand side has multiple speakers

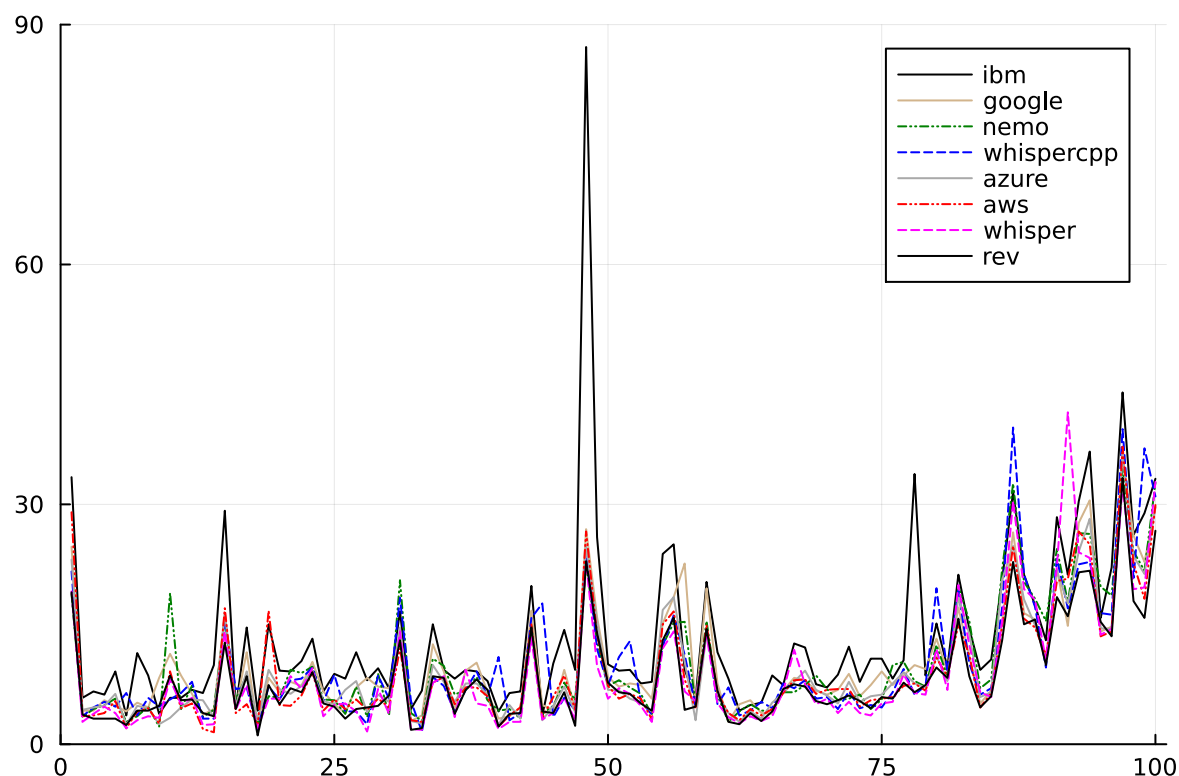


```

1 begin
2   wers_sorted_by_nsp = sort(wers, [:nsp])
3   plot(wers_sorted_by_nsp.ibm,label="ibm", linestyle=:solid, color=:black)
4   plot!(wers_sorted_by_nsp.google,label="google", linestyle=:solid, color=:tan)
5   plot!(wers_sorted_by_nsp.nemo,label="nemo", linestyle=:dashdotdot, color=:green)
6   plot!(wers_sorted_by_nsp.whispercpp,label="whispercpp", linestyle=:dash,
7         color=:blue)
8   plot!(wers_sorted_by_nsp.azure,label="azure", linestyle=:solid, color=:darkgray)
9   plot!(wers_sorted_by_nsp.aws,label="aws",xlim=(0,101),ylim=(0,90),
10        linestyle=:dashdotdot,color=:red)
11  plot!(wers_sorted_by_nsp.whisper,label="whisper", linestyle=:dash,
12        color=:magenta)
13  plot!(wers_sorted_by_nsp.rev,label="rev", linestyle=:solid,color=:black)
14  vline!([68],label="68")
15 end

```

Rather than the number of speakers directly, WER seems to be affected by overlapping speech, where systems will drop some of the speech. Here we plot WER sorted by the amount of overlapped speech.

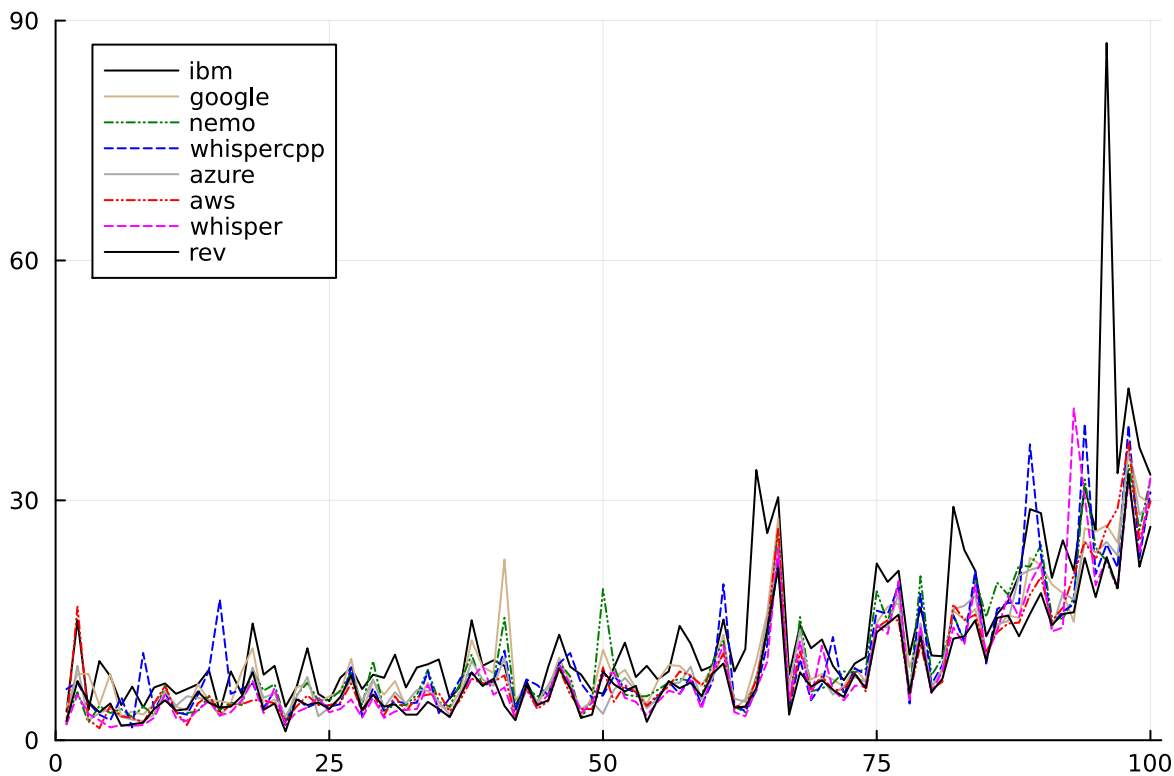


```

1 begin
2   wers_sorted_by_ov = sort(wers, [:overlap])
3   plot(wers_sorted_by_ov.ibm,label="ibm", linestyle=:solid, color=:black)
4   plot!(wers_sorted_by_ov.google,label="google", linestyle=:solid, color=:tan)
5   plot!(wers_sorted_by_ov.nemo,label="nemo", linestyle=:dashdotdot, color=:green)
6   plot!(wers_sorted_by_ov.whispercpp,label="whispercpp", linestyle=:dash,
7     color=:blue)
8   plot!(wers_sorted_by_ov.azure,label="azure", linestyle=:solid, color=:darkgray)
9   plot!(wers_sorted_by_ov.aws,label="aws",xlim=(0,101),ylim=(0,90),
10    linestyle=:dashdotdot,color=:red)
11  plot!(wers_sorted_by_ov.whisper,label="whisper", linestyle=:dash, color=:magenta)
12  plot!(wers_sorted_by_ov.rev,label="rev", linestyle=:solid,color=:black)
13 end

```

Transcribers would annotate unintelligible regions, meaning regions they couldn't understand, or were making a best guess. These regions are almost always errors for the system. Here we plot WERs sorted by a count of these unintelligible regions.

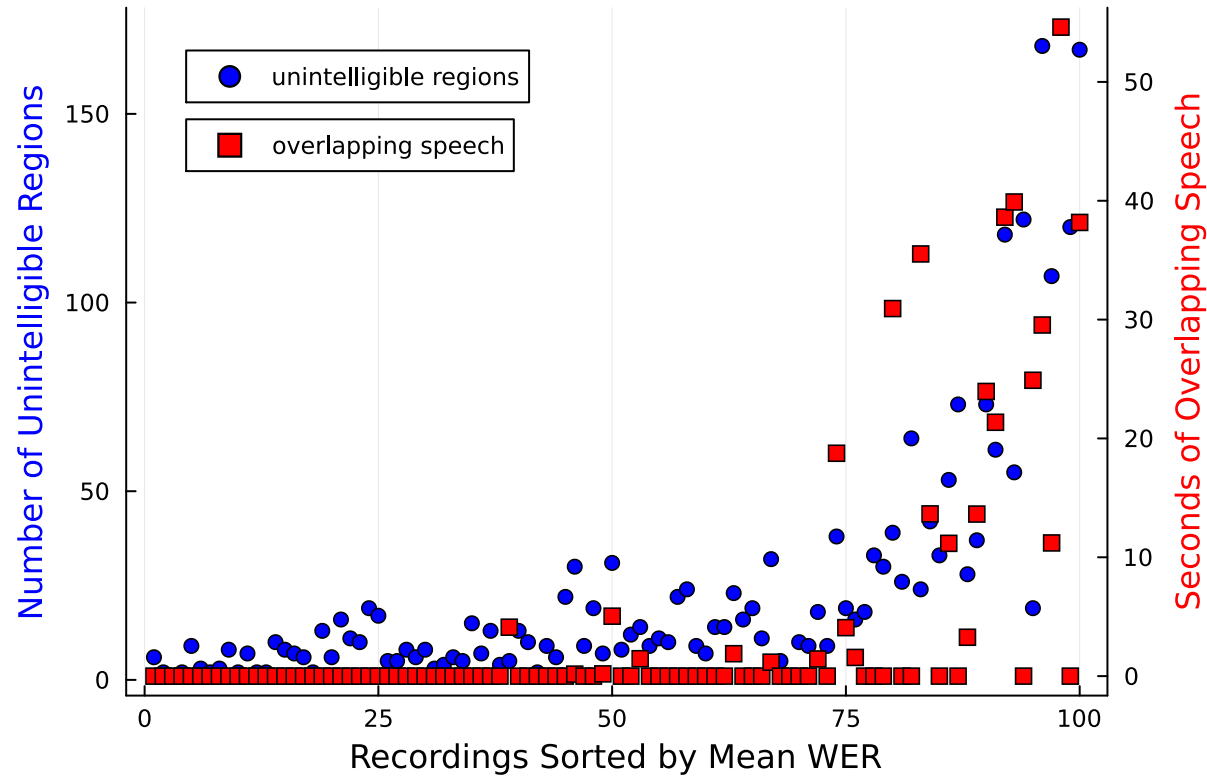


```

1 begin
2   wers_sorted_by_un = sort(wers, [:unintelligible])
3   plot(wers_sorted_by_un.ibm,label="ibm", linestyle=:solid, color=:black)
4   plot!(wers_sorted_by_un.google,label="google", linestyle=:solid, color=:tan)
5   plot!(wers_sorted_by_un.nemo,label="nemo", linestyle=:dashdotdot, color=:green)
6   plot!(wers_sorted_by_un.whispercpp,label="whispercpp", linestyle=:dash,
7     color=:blue)
8   plot!(wers_sorted_by_un.azure,label="azure", linestyle=:solid, color=:darkgray)
9   plot!(wers_sorted_by_un.aws,label="aws",xlim=(0,101),ylim=(0,90),
10    linestyle=:dashdotdot,color=:red)
11  plot!(wers_sorted_by_un.whisper,label="whisper", linestyle=:dash, color=:magenta)
12  plot!(wers_sorted_by_un.rev,label="rev", linestyle=:solid,color=:black)
13 end

```

Overlap and unintelligibility account for much of the higher WERs. Here we plot both, sorted by mean WER. Note that the y values of the two plots are completely different: the line plot is the count of unintelligible regions in the transcripts, and the scatter plot is the amount of overlapping speech in seconds, usually zero.



```
1 begin
2   scatter(wers_sorted_by_mean.unintelligible, label="unintelligible regions",
3   legend=(0.15,.9), xaxis="Recordings Sorted by Mean WER", ylabel="Number of
4   Unintelligible Regions", yguidefontcolor=:blue,color=:blue)
5   scatter!(twinx(),wers_sorted_by_mean.overlap, label="overlapping speech",
6   color=:red, legend=(0.15,0.8), ylabel="Seconds of Overlapping
7   Speech",yguidefontcolor=:red,m=:rect)
8 end
```

unov_model = StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}}, GLM.DensePredCho

mean ~ 1 + unintelligible + overlap

Coefficients:

	Coef.	Std. Error	t	Pr(> t)	Lower 95%	Upper 95%
(Intercept)	5.62327	0.417809	13.46	<1e-23	4.79404	6.45251
unintelligible	0.149611	0.0133252	11.23	<1e-18	0.123164	0.176057
overlap	0.134205	0.0442777	3.03	0.0031	0.0463257	0.222084

```
1 unov_model = lm(@formula(mean ~ unintelligible + overlap), wers_with_mean)
```

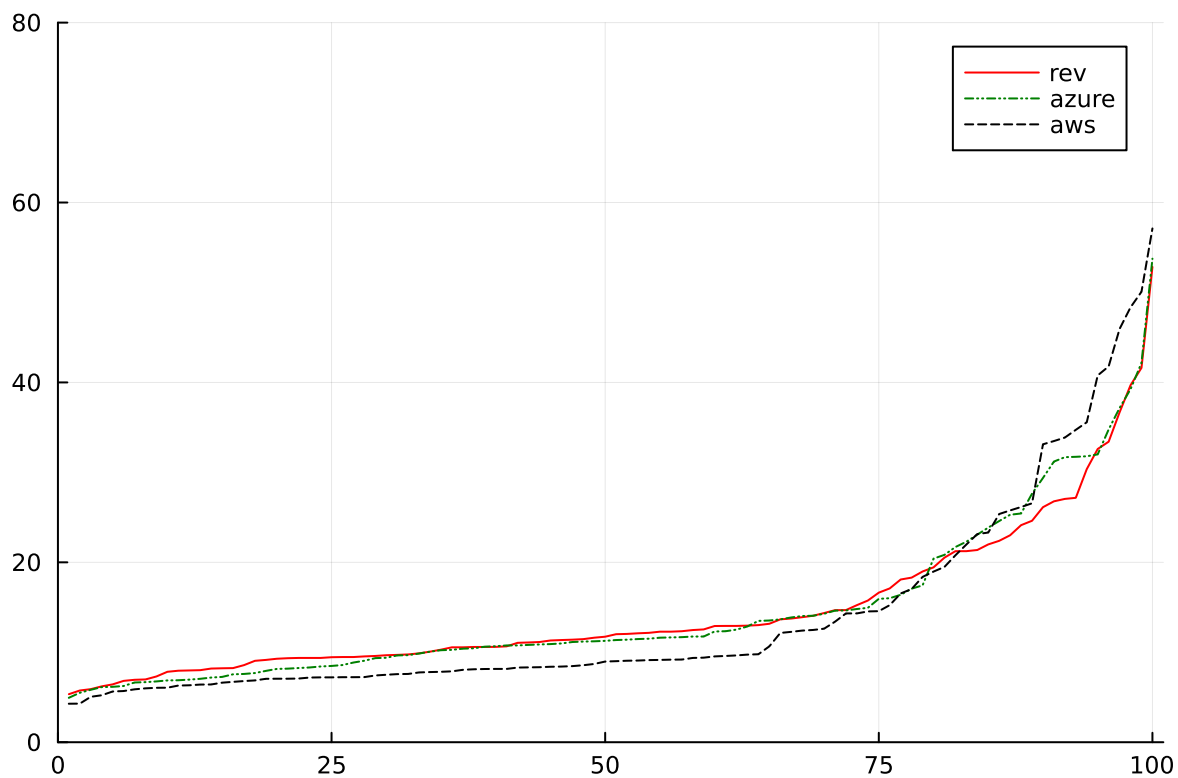
Diarization Error Rates

der =

	file	aws	azure	rev
1	"AndrewsEx1"	16.54	13.53	23.01
2	"AntinEx1"	20.78	25.43	27.17
3	"Ashbery1Ex1"	9.59	10.62	10.59
4	"Ashbery2Ex1"	8.15	11.36	12.47
5	"Ashbery3Ex1"	8.05	10.87	11.73
6	"Ashbery4Ex1"	21.98	31.79	24.62
7	"Ashbery5Ex1"	8.41	8.18	6.82
8	"Ashbery6Ex1"	9.08	11.47	9.37
9	"Ashbery7Ex1"	8.15	8.48	9.37
10	"Ashbery8Ex1"	8.97	11.27	12.54
more				
100	"YauEx1"	7.06	7.69	9.47

```
1 der = CSV.read("der.tsv", DataFrame, delim="\t")
2
```

DERs sorted separately



```
1 begin
2   plot(sort(der.rev),label="rev",linestyle=:solid,color=:red,xlim=(0,101),ylim=
3     (0,80))
4   plot!(sort(der.azure),label="azure", linestyle=:dashdotdot, color=:green)
5   plot!(sort(der.aws),label="aws", linestyle=:dash, color=:black)
6 end
```

What about the outliers?

IBM specifically

what happened with ibm?

There's one extreme outlier, *JorisEx1*, where most words are missing. This recording has a lot of distortion/feedback.

	file	aws	azure	google	ibm	nemo	rev	whisper	whispercpp
1	"JorisEx1"	26.6	24.8	26.9	87.2	22.4	22.9	22.7	24.5

```
1 wers[ wers.file .== "JorisEx1", :]
```

Another outlier is *CorriganEx1*

	file	aws	azure	google	ibm	nemo	rev	whisper	whispercpp
1	"CorriganEx1"	7.6	7.6	9.9	33.8	7.9	6.5	6.4	6.2

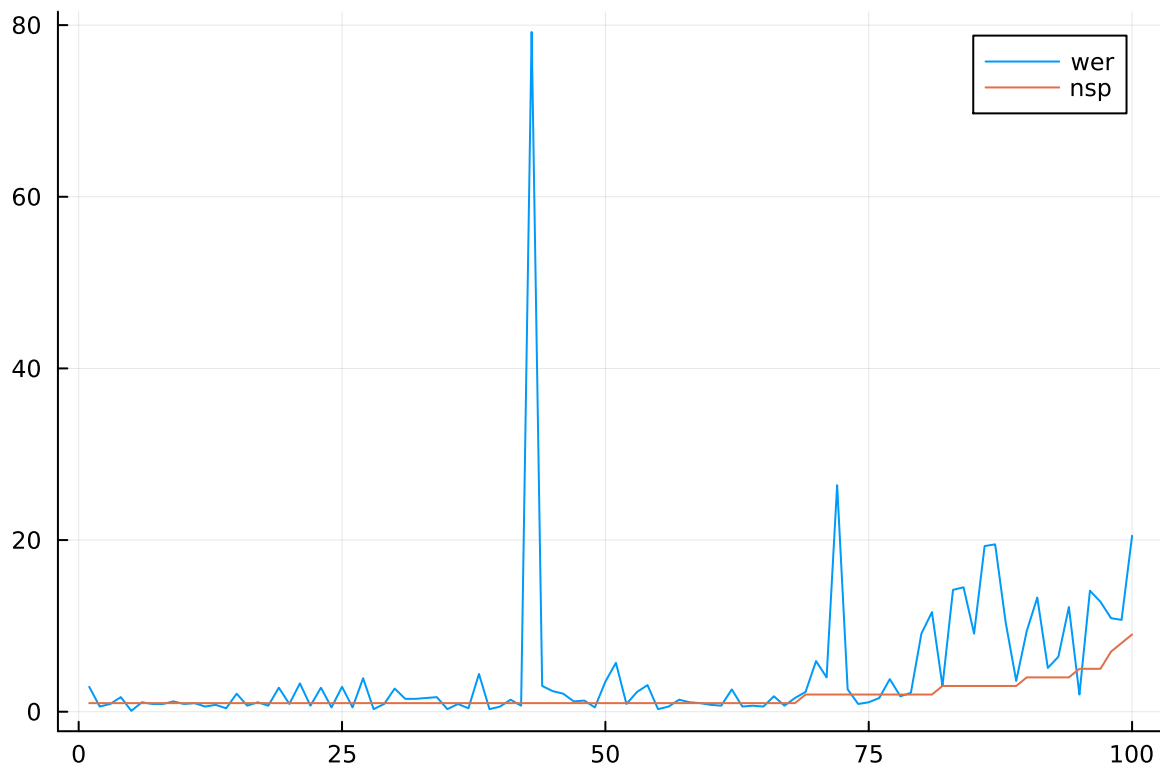
```
1 wers[wers.file .== "CorriganEx1", :]
```

this file is unremarkable, but also has a lot of deletions. E.g., there's a 15s region from 100s to 115s that's missing, which is where a second speaker joins.

	file	aws	azure	google	ibm	nemo	rev	whisper	whispercpp
1	"CorriganEx1"	1.6	1.4	3.0	26.4	3.3	1.3	1.3	1.4

```
1 wers_d[ wers_d.file .== "CorriganEx1", :]
```

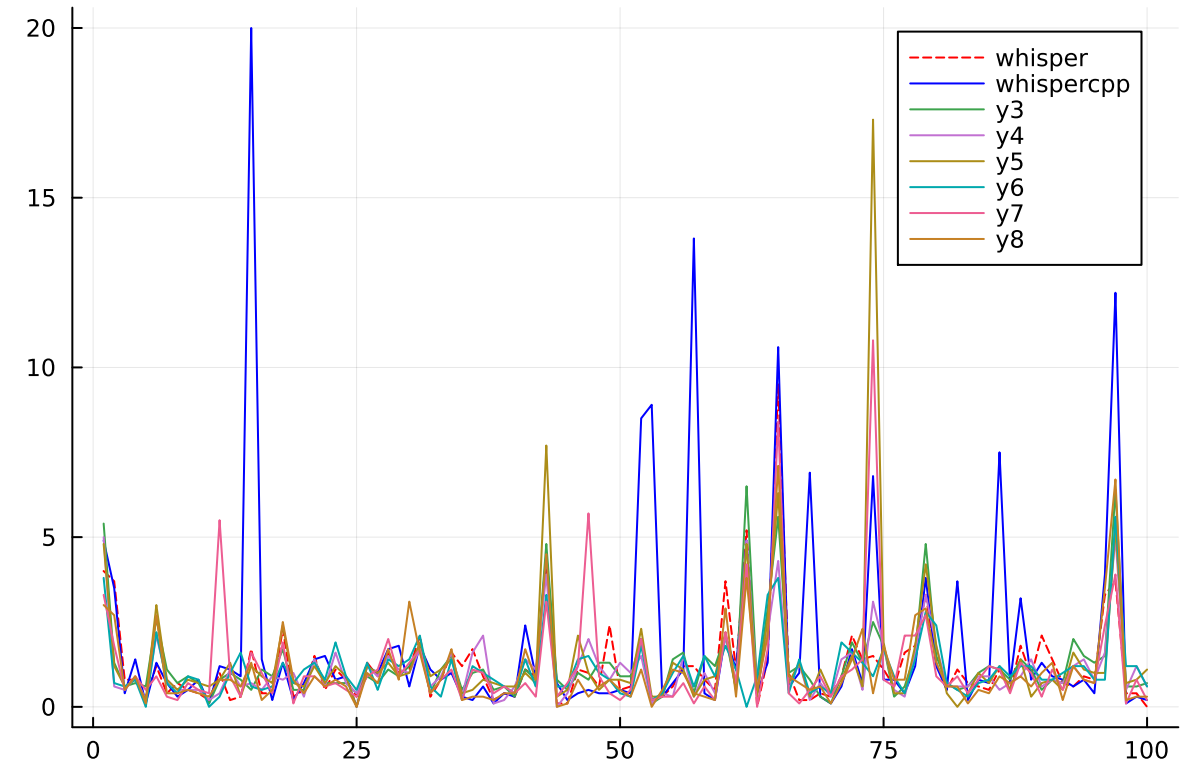
so IBM seems to have an issue with deletions, maybe due to speakers. sorting by number of speakers shows some effect. The first 68 have a single speaker. *CorriganEx1* has two speakers, so this doesn't seem totally explanatory for it's high DER.



```
1 begin
2   wers_dn = sort(wers_d, [:nsp])
3   plot(wers_dn.ibm, label="wer")
4   # plot!(wers_d.ibm)
5   plot!(wers_dn.nsp, label="nsp")
6   # plot!(wers.snr, label="snr")
7 end
```

Whisper and Whisper cpp specifically

whispercpp (dark blue solid) stands out among IERs due to hallucinations, but *whisper* (red dashed) does not. *whispercpp* doesn't have the new options to limit hallucinations



```
1 begin
2   wm = wers_i
3   plot(wm.whisper, label="whisper", color=:red, linestyle=:dash)
4   plot!(wm.whispercpp, label="whispercpp", color=:blue)
5   plot!(wm.aws)
6   plot!(wm.azure)
7   plot!(wm.google)
8   plot!(wm.ibm)
9   plot!(wm.nemo)
10  plot!(wm.rev)
11 end
```

Google specifically

PhillyTalks10Ex1 is an outlier for google with a very high IER. There's a long string of individual digits inserted during a period of silence (no speech), similar to a hallucination in whisper, although it's only digits.

	file	aws	azure	google	ibm	nemo	rev	whisper	whispy
1	"PhillyTalks10Ex1"	2.5	3.1	17.3	0.9	10.8	0.4	1.5	6.8

```
1 wers_i[wers_i.file .== "PhillyTalks10Ex1", :]
```

